LAPORAN MATERI DAN PROGRAM IMAGE ENHANCEMENT UNTUK NORMALISASI HISTOGRAM DARI CITRA

Dosen Pengampu:

Afdhal Dinilhak, M.Kom.

Muhammad Luthfi Kautsar Rizata

2311532020

PROGRAM STUDI INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

SEMESTER GENAP 2025/2026

A. Materi Pengantar (Resume)

1. Konsep Dasar Image Enhancement

Image enhancement merupakan salah satu tahapan dalam pengolahan citra digital yang berfungsi sebagai proses awal sebelum digunakan untuk analisis lebih lanjut. Tujuan utama dari image enhancement adalah memperoleh citra dengan kualitas yang lebih baik dibandingkan citra awal, sehingga interpretasi terhadap citra tersebut dapat dilakukan dengan lebih akurat.

Naiknya kebutuhan terhadap image enhancement muncul akibat berbagai faktor yang dapat menurunkan kualitas citra, seperti keberadaan noise, kondisi pencahayaan yang tidak ideal (terlalu terang atau terlalu gelap), ketajaman citra yang kabur, cacat saat proses akuisisi, serta distorsi geometrik yang disebabkan oleh keterbatasan lensa atau sudut pengambilan gambar yang tidak tepat.

Berdasarkan domain operasinya, metode image enhancement dapat dikategorikan menjadi dua kategori utama. Pertama, image enhancement dalam domain spasial yang melakukan manipulasi secara langsung terhadap pixel-pixel penyusun citra. Kedua, image enhancement dalam domain frekuensi yang bekerja dengan mengubah citra dari domain spasial ke domain frekuensi terlebih dahulu, kemudian melakukan manipulasi terhadap nilai-nilai frekuensi tersebut. Masing-masing domain memiliki tujuan spesifik karena tidak semua jenis perbaikan citra dapat dilakukan dalam domain spasial.

2. Image Enhancement dalam Domain Spasial

Dalam domain spasial, proses peningkatan kualitas citra dapat dinyatakan secara matematis melalui persamaan

$$g(x, y) = T[f(x, y)]$$

di mana:
$f(x, y)$ = citra input .
$g(x, y)$ = adalah citra output
T = operator yang bekerja terhadap citra input.

Operator T dapat beroperasi pada tiga tingkatan yang berbeda, yaitu pada satu pixel tunggal (point processing), pada sekelompok pixel yang bertetangga (local processing), atau pada keseluruhan pixel di dalam citra (global processing).

Point processing merupakan metode pengolahan citra yang dilakukan per pixel secara individu tanpa melibatkan pixel-pixel tetangga di sekitarnya. Metode ini menjadi dasar bagi berbagai teknik peningkatan kualitas citra karena kesederhanaan dan efisiensi komputasinya.

Thresholding adalah teknik point processing yang banyak digunakan untuk segmentasi, yaitu proses pemisahan objek dari latar belakang citra. Operasi thresholding didefinisikan sebagai:

$$f(x,y)' = \begin{cases} a_1, & f(x,y) < T \\ a_2, & f(x,y) \geq T \end{cases}$$

Apabila $a_1 = 0$ dan $a_2 = 1$, maka transformasi tersebut akan mengubah citra hitam-putih menjadi citra biner. Sebagai contoh, dengan nilai ambang $T = 128$, pixel dengan intensitas di bawah 128 akan diubah menjadi 0 (hitam), sedangkan pixel dengan intensitas 128 atau lebih akan diubah menjadi 1 (putih).

Image negative dihasilkan dengan mengurangi nilai intensitas setiap pixel dari nilai maksimum keabuan yang mungkin. Untuk citra grayscale 8-bit dengan rentang nilai 0-255, operasi ini dinyatakan sebagai $f(x,y)' = 255 - f(x,y)$, sedangkan untuk citra 7-bit dengan rentang 0-127, operasinya adalah $f(x,y)' = 127 - f(x,y)$. Hasilnya adalah citra dengan hubungan intensitas terbalik dari citra asli.

Image contrast berkaitan dengan sebaran terang dan gelap dalam sebuah citra. Citra dengan kontras rendah dicirikan oleh dominasi warna terang atau gelap saja, sementara citra dengan kontras baik memiliki komposisi gelap dan terang yang tersebar merata. Contrast stretching merupakan teknik untuk mengubah kontras citra melalui fungsi transformasi yang memetakan nilai intensitas pixel asli ke nilai intensitas baru.

Operasi aritmetika pada citra mencakup berbagai manipulasi matematis. Penjumlahan dan pengurangan antar citra dinyatakan sebagai $C(x,y) = A(x,y) \pm B(x,y)$, sedangkan perkalian antar citra adalah $C(x,y) = A(x,y) \times B(x,y)$. Operasi penjumlahan citra dapat digunakan untuk mengurangi noise melalui proses rata-rata dari beberapa citra yang sama, dirumuskan sebagai $f'(x,y) = \frac{1}{2}\{f_1(x,y) + f_2(x,y)\}$. Pengurangan citra (image subtraction) bermanfaat untuk mengekstraksi bagian tertentu dari citra, seperti dalam aplikasi medis untuk mendapatkan jaringan toraks dari citra rontgen. Perkalian citra dengan matriks koreksi sering digunakan untuk tujuan kalibrasi.

Operasi aritmetika juga dapat melibatkan skalar. Penjumlahan citra A dengan skalar c menghasilkan $B(x,y) = A(x,y) + c$ yang membuat citra lebih terang, sedangkan pengurangan dengan skalar $B(x,y) = A(x,y) - c$ menghasilkan citra lebih gelap. Operasi ini banyak digunakan untuk pencerahan citra. Perkalian citra dengan skalar $B(x,y) = c \times A(x,y)$ digunakan untuk kalibrasi kecerahan, sementara pembagian dengan skalar digunakan untuk normalisasi kecerahan.

Operasi boolean memegang peranan penting dalam pemrosesan morfologi pada citra biner, contohnya:

- Operasi AND dinyatakan sebagai $C(x, y) = A(x, y) \wedge B(x, y)$
- operasi OR sebagai $C(x, y) = A(x, y) \vee B(x, y)$
- operasi NOT sebagai $C(x, y) = \neg A(x, y)$
- operasi XOR sebagai $C(x, y) = A(x, y) \oplus B(x, y)$

Operasi geometri mencakup berbagai transformasi spasial seperti

- translasi (pergeseran posisi)
- rotasi (pemutaran citra)
- penskalaan (perubahan ukuran)
- dilatasi (pelebaran)
- distorsi
- image morphing (transformasi bentuk antar citra)

3. Analisis Histogram Citra

Histogram citra merupakan representasi grafis yang menunjukkan distribusi nilai intensitas pixel dalam suatu citra digital. Histogram menjadi alat bantu yang penting dalam pengolahan citra, baik untuk analisis kualitatif maupun kuantitatif, karena dapat mengungkapkan berbagai karakteristik penting seperti kecerahan dan kontras dari sebuah gambar atau citra.

Untuk citra dengan L level nilai keabuan $[0, L-1]$, histogram dihitung dengan menentukan frekuensi kemunculan setiap nilai keabuan j, yaitu jumlah pixel yang memiliki nilai keabuan tersebut untuk $j = 0,1,2, \dots, L-1$. Pada citra berwarna dengan komponen R, G, dan B, histogram dibuat secara terpisah untuk setiap kanal warna.

Normalisasi histogram dilakukan dengan membagi frekuensi setiap nilai keabuan dengan jumlah seluruh pixel dalam citra, dirumuskan sebagai:

$$h_i = \frac{n_i}{n}$$

di mana:

- n_i = jumlah pixel yang memiliki derajat keabuan i
- n = jumlah seluruh pixel dalam citra untuk $i = 0,1, \dots, L-1$

- h_i = berada dalam selang 0 sampai 1 dan menyatakan peluang (probability) munculnya pixel dengan nilai keabuan i, atau dinotasikan sebagai $p(i)$.
-

a. Histogram Ternormalisasi

Histogram ternormalisasi memiliki beberapa sifat matematis penting, yaitu:

- Jumlah seluruh nilai h_i sama dengan 1, yaitu $\sum_{i=0}^{L-1} h_i = 1$
- Peluang suatu pixel memiliki derajat keabuan lebih kecil atau sama dengan nilai tertentu dinyatakan sebagai $P(i \leq j) = \sum_{i=0}^{j} h_i$ untuk $0 \leq i \leq j$.

Puncak-puncak dalam histogram menunjukkan intensitas pixel yang dominan, sementara lebar puncak mengindikasikan rentang kontras citra. Berdasarkan karakteristik histogramnya, citra dapat diklasifikasikan menjadi empat tipe:

- **Citra gelap (underexposed)** memiliki komponen histogram yang terkonsentrasi pada nilai-nilai grayscale rendah, yaitu sisi kiri histogram.
- **Citra terang (overexposed)** menunjukkan konsentrasi histogram pada nilai-nilai grayscale tinggi atau sisi kanan histogram.
- **Citra kontras rendah (low contrast)** memiliki histogram yang sempit dan terpusat di pertengahan nilai-nilai grayscale.
- **Citra kontras tinggi (high contrast)** dicirikan oleh histogram yang tersebar merata di seluruh rentang nilai grayscale dengan distribusi pixel yang relatif uniform, tanpa garis vertikal yang jauh lebih tinggi dari yang lain.

Citra dengan kualitas baik umumnya memiliki histogram yang mengisi seluruh daerah derajat keabuan secara penuh dengan distribusi yang merata pada setiap nilai intensitas pixel.

b. Ukuran Statistika dari Histogram

Nilai rata-rata (mean value) derajat keabuan dihitung dengan rumus:

$$u = \sum_{k} k \cdot h(k)$$

Nilai ini menunjukkan rata-rata nilai keabuan dari seluruh pixel dalam citra.

Variansi memberikan ukuran distribusi nilai histogram di sekitar nilai rata-rata, dirumuskan sebagai:

$$v = \sum_{k} (k - u)^2 h(k)$$

Standar deviasi yang merupakan akar kuadrat dari variansi, $d = \sqrt{v}$, menunjukkan jarak rata-rata semua pixel ke nilai rata-rata pada sumbu nilai keabuan. Variansi dan standard deviasi histogram memberikan informasi penting tentang rata-rata kontras citra, di mana semakin tinggi nilai

variansi, maka semakin tinggi pula kontras citra tersebut.

c. Histogram Equalization

Histogram equalization merupakan teknik peningkatan kualitas citra yang bertujuan mengubah distribusi gray level agar kontrasnya lebih menyebar pada seluruh kisaran nilai yang tersedia (misalnya 0-255 untuk citra 8-bit). Prinsip kerja teknik ini adalah memberikan jarak yang lebih lebar antar gray level yang sering muncul, sementara gray level yang jarang muncul dapat dirapatkan jaraknya dengan gray level sebelumnya. Hasil dari histogram equalization adalah histogram baru yang selalu mencapai nilai maksimal keabuan (255) dengan distribusi yang lebih merata, sehingga meningkatkan kontras visual citra secara keseluruhan.

B. Resume Tugas 1 (Operasi Perkalian/Pembagian Citra Dengan Skalar)

Untuk membuat tugas implementasi operasi perkalian dan pembagian citra dengan skalar di Collab, kita harus melakukan import pada modul:

- cv2 adalah library dari OpenCV yang digunakan untuk mengolah gambar dan video, seperti membaca citra, filtering, deteksi objek, dan operasi pengolahan citra lainnya.
- numpy digunakan untuk mengelola data dalam bentuk array atau matriks. Dalam image processing, gambar direpresentasikan sebagai matriks piksel sehingga operasi perhitungan menjadi lebih cepat dan efisien.
- matplotlib.pyplot dipakai untuk menampilkan gambar atau grafik hasil pengolahan agar bisa divisualisasikan dengan mudah.

```
[1]
✓ 0s
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

Selanjutnya, kita membuat fungsi show_images(images, titles) yang digunakan untuk menampilkan beberapa gambar sekaligus dalam satu tampilan agar mudah dibandingkan.

- plt.figure(figsize=(15, 5))
  Mengatur ukuran area tampilan gambar supaya lebih lebar dan rapi.

- for i in range(len(images)):
  Perulangan untuk menampilkan setiap gambar yang ada di dalam list images.

- plt.subplot(1, len(images), i+1)
  Membagi tampilan menjadi beberapa kolom dalam satu baris dimana jumlah kolom menyesuaikan jumlah gambar.

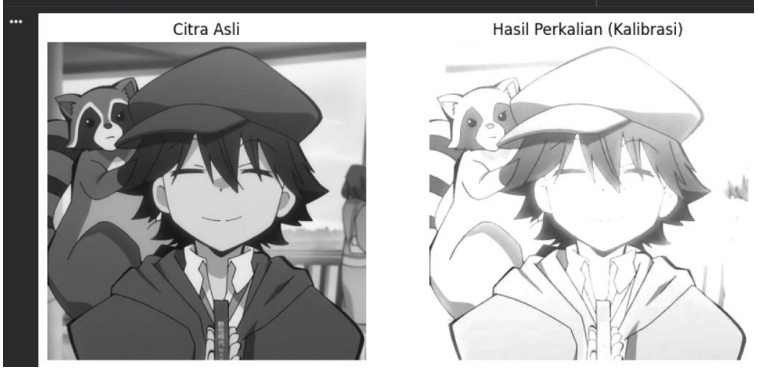
- plt.imshow(images[i], cmap='gray')
  Menampilkan gambar ke-i. Parameter cmap='gray' sehingga gambar ditampilkan dalam format grayscale.
- plt.title(titles[i])
  Memberi judul pada setiap gambar sesuai urutan.
- plt.axis('off')
  Menghilangkan tampilan sumbu koordinat supaya gambar terlihat lebih bersih.
- plt.show()
  Menampilkan seluruh gambar yang sudah disusun.

```
def show_images(images, titles):
    plt.figure(figsize=(15, 5))
    for i in range(len(images)):
        plt.subplot(1, len(images), i+1)
        plt.imshow(images[i], cmap='gray')
        plt.title(titles[i])
        plt.axis('off')
    plt.show()
```

Kode selanjutnya digunakan untuk melakukan pembacaan citra grayscale lalu meningkatkan kecerahan citra yang mana merupakan implementasi operasi perkalian citra dengan skalar. Terlihat citra output yang setelah dikalibrasi terlihat lebih terang daripada citra aslinya.

- img = cv2.imread('Ranpo Edogawa.jpg', cv2.IMREAD_GRAYSCALE)
  Baris ini membaca gambar dalam format grayscale, sehingga setiap piksel hanya memiliki satu nilai intensitas dari 0 (hitam) sampai 255 (putih).

```
img = cv2.imread('Ranpo Edogawa.jpg', cv2.IMREAD_GRAYSCALE)
c_mul = 3
img_multiplied = np.clip(img.astype(np.float32) * c_mul, 0, 255).astype(np.uint8)
show_images([img, img_multiplied], ['Citra Asli', 'Hasil Perkalian (Kalibrasi)'])
```



- c_mul = 3
  Variabel ini adalah konstanta pengali untuk menaikkan intensitas piksel. Semakin besar nilainya, gambar akan terlihat semakin terang.
- img.astype(np.float32)
  Mengubah tipe data piksel menjadi float agar aman saat proses perkalian dan mencegah error overflow.
- * c_mul
  Setiap nilai piksel dikalikan konstanta sehingga intensitas cahaya meningkat.

- np.clip(..., 0, 255)
  Membatasi hasil perkalian agar tetap berada pada rentang nilai piksel yang valid (0–255).
- .astype(np.uint8)
  Mengubah kembali tipe data menjadi format citra standar 8-bit.
- show_images([img, img_multiplied], ['Citra Asli', 'Hasil Perkalian (Kalibrasi)'])
  Menampilkan gambar asli dan gambar hasil perkalian secara berdampingan supaya perbedaannya terlihat jelas.

Selain itu, kita bisa melakukan normalisasi intensitas citra dengan cara mengimplementasikan kode yang melakukan pembagian pada setiap nilai piksel menggunakan suatu konstanta atau skalar sehingga citra output terlihat lebih gelap daripada citra aslinya.

- c_div = 100.0
  Variabel ini adalah konstanta pembagi. Nilai ini menentukan seberapa besar intensitas piksel akan diperkecil.
- img.astype(np.float32)
  Mengubah tipe data citra menjadi float agar proses pembagian lebih akurat dan aman dari error.

- / c_div
  Setiap nilai piksel dibagi dengan konstanta sehingga intensitasnya menurun dan citra terlihat lebih gelap.
- np.clip(..., 0, 255)
  Membatasi nilai piksel agar tetap berada pada rentang valid citra grayscale.
- .astype(np.uint8)
  Mengubah kembali tipe data menjadi format citra standar 8-bit.
- show_images([img, img_divided], ['Citra Asli', 'Hasil Pembagian (Normalisasi)'])
  Menampilkan citra asli dan citra hasil normalisasi secara berdampingan untuk melihat perbedaannya.

C. Image Enhancement untuk Normalisasi Histogram

Untuk melakukan implementasi kode untuk image enhancement menggunakan normalisasi histogram. Pertama-tama, import beberapa library seperti:

- NumPy: untuk melakukan komputasi numerik dan memanipulasi data citra dalam bentuk array matriks sehingga operasi matematis dapat dilakukan dengan efisien.
- Matplotlib: untuk menampilkan citra serta memvisualisasikan histogram distribusi intensitas piksel.
- Pillow (PIL) digunakan untuk membaca file citra dan mengubahnya ke format yang dapat diproses lebih lanjut.

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
```

Setelah itu, kita bisa menyimpan gambar yang sudah di-upload ke dalam collab yang disimpan ke dalam variabel img. Dalam variabel ini, ada fungsi Image.open() dari library Pillow (PIL) yang digunakan untuk membuka file citra dari penyimpanan dengan nama *Dazai Osamu.jpg*. Selanjutnya, fungsi .convert("L") mengubah citra menjadi mode grayscale, yaitu citra keabuan yang hanya memiliki satu kanal intensitas sehingga lebih sederhana untuk proses pengolahan. Setelah itu, citra dikonversi menjadi array menggunakan NumPy melalui fungsi np.array(), sehingga setiap piksel dapat direpresentasikan sebagai nilai numerik dalam matriks agar bisa diproses menggunakan operasi matematis.

```
img = Image.open("Dazai Osamu.jpg").convert("L")
img_array = np.array(img)
```

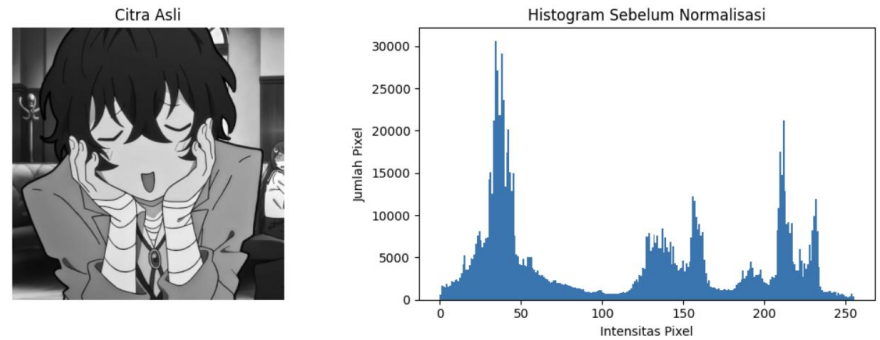
Setelah itu, citra yang sudah dikonversi menjadi grayscale akan terlihat berwarna abu-abu seperti pada gambar. Sebelum dilakukan normalisasi, histogram citra didominasi oleh intensitas pixel yang berada di bawah nilai 50

• fig, ax = plt.subplots(1, 2, figsize=(11,4))
Baris ini membuat sebuah figure dengan dua subplot yang disusun dalam satu baris dan dua kolom. Parameter figsize mengatur ukuran tampilan agar gambar dan histogram terlihat proporsional.

```
fig, ax = plt.subplots(1, 2, figsize=(11,4))

# Subplot 1 — Image
ax[0].imshow(img_array, cmap='gray')
ax[0].set_title("Citra Asli")
ax[0].axis('off')

# Subplot 2 — Histogram
ax[1].hist(img_array.flatten(), bins=256)
ax[1].set_title("Histogram Sebelum Normalisasi")
ax[1].set_xlabel("Intensitas Pixel")
ax[1].set_ylabel("Jumlah Pixel")

plt.tight_layout()
plt.show()
```

• ax[0].imshow(img_array, cmap='gray')
Menampilkan citra grayscale dari variabel img_array pada subplot pertama. Parameter cmap='gray' memastikan citra ditampilkan dalam tingkat keabuan.

• ax[0].set_title("Citra Asli")
Memberikan judul pada subplot pertama untuk menjelaskan bahwa gambar yang ditampilkan adalah citra asli sebelum diproses.



• ax[0].axis('off')
Menyembunyikan sumbu koordinat pada gambar sehingga tampilan citra menjadi lebih rapi dan fokus pada visual gambar saja.

• ax[1].hist(img_array.flatten(), bins=256)
Menampilkan histogram distribusi intensitas piksel pada subplot kedua. Fungsi flatten() mengubah matriks citra menjadi array satu dimensi agar dapat dihitung frekuensi tiap nilai piksel. Parameter bins=256 menunjukkan jumlah tingkat intensitas grayscale dari 0 hingga 255.

• ax[1].set_title("Histogram Sebelum Normalisasi")
Memberikan judul pada histogram untuk menjelaskan bahwa grafik menunjukkan distribusi intensitas sebelum proses normalisasi.

• ax[1].set_xlabel("Intensitas Pixel")
Memberi label pada sumbu horizontal yang menunjukkan nilai intensitas piksel.

• ax[1].set_ylabel("Jumlah Pixel")
Memberi label pada sumbu vertikal yang menunjukkan jumlah piksel pada setiap tingkat intensitas.

• plt.tight_layout()
Mengatur jarak antar subplot secara otomatis agar tidak saling bertumpuk dan terlihat lebih rapi

• plt.show()
Menampilkan seluruh hasil visualisasi subplot ke layar output.

Selanjutnya proses normalisasi histogram dilakukan dengan tujuan untuk memperlebar rentang intensitas piksel sehingga kontras citra menjadi lebih jelas. Normalisasi ini bertujuan agar nilai piksel tersebar merata dari intensitas minimum hingga maksimum (0–255), sehingga detail citra lebih terlihat.

```
I_min = img_array.min()
I_max = img_array.max()

norm_img = (img_array - I_min) / (I_max - I_min) * 255
norm_img = norm_img.astype(np.uint8)
```

• I_min = img_array.min()
Baris ini digunakan untuk mencari nilai intensitas piksel paling kecil pada citra grayscale. Nilai minimum ini nantinya menjadi batas bawah dalam proses normalisasi.

• I_max = img_array.max()
Baris ini berfungsi untuk mencari nilai intensitas piksel paling besar pada citra grayscale. Nilai maksimum ini menjadi batas atas dalam proses normalisasi.

• norm_img = (img_array - I_min) / (I_max - I_min) * 255
Baris ini merupakan inti dari proses normalisasi histogram. Setiap nilai piksel pada citra dikurangi dengan intensitas minimum agar rentangnya dimulai dari nol. Kemudian hasilnya dibagi dengan selisih intensitas maksimum dan minimum untuk menyesuaikan skala distribusi piksel. Setelah itu dikalikan 255 agar rentang intensitas kembali sesuai standar citra grayscale, yaitu 0 sampai 255.

• norm_img = norm_img.astype(np.uint8)
Baris ini mengubah tipe data piksel menjadi unsigned 8-bit integer agar sesuai dengan format standar citra digital. Konversi ini penting supaya nilai piksel tidak berupa pecahan desimal dan dapat ditampilkan sebagai gambar dengan benar.

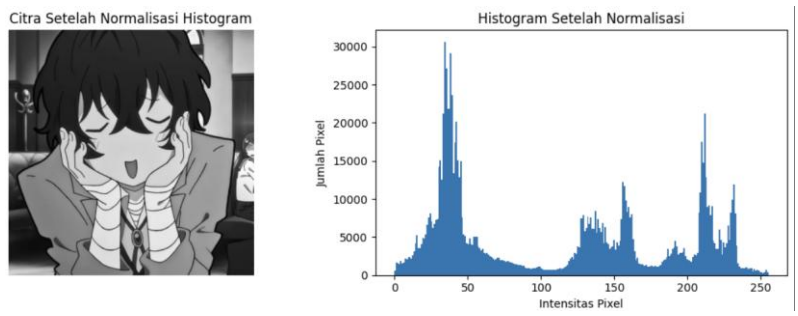
Setelah proses normalisasi histogram selesai, citra hasil normalisasi kemudian ditampilkan beserta histogram terbarunya untuk melihat perubahan distribusi intensitas piksel. Jika dilihat sekilas citra setelah normalisasi, masih terlihat mirip dengan citra asli sebelum dilakukan normalisasi

• fig, ax = plt.subplots(1, 2, figsize=(11,4))
Baris ini membuat sebuah figure dengan dua subplot yang disusun dalam satu baris dan dua kolom. Parameter figsize mengatur ukuran tampilan agar citra dan histogram terlihat lebih jelas dan proporsional.

• ax[0].imshow(norm_img, cmap='gray')
Menampilkan citra hasil normalisasi histogram pada subplot pertama dalam bentuk grayscale sehingga perubahan kontras dapat terlihat dengan jelas.

```
fig, ax = plt.subplots(1, 2, figsize=(11,4))

# Subplot 1 — Normalized Image
ax[0].imshow(norm_img, cmap='gray')
ax[0].set_title("Citra Setelah Normalisasi Histogram")
ax[0].axis('off')

# Subplot 2 — Histogram Normalized
ax[1].hist(norm_img.flatten(), bins=256)
ax[1].set_title("Histogram Setelah Normalisasi")
ax[1].set_xlabel("Intensitas Pixel")
ax[1].set_ylabel("Jumlah Pixel")

plt.tight_layout()
plt.show()
```

• ax[0].set_title("Citra Setelah Normalisasi Histogram")
Memberikan judul pada subplot pertama untuk menjelaskan bahwa citra yang ditampilkan merupakan c itra hasil normalisasi histogram.



• ax[0].axis('off')
Menyembunyikan sumbu koordinat agar tampilan citra lebih rapi dan fokus pada visual gambar.

• ax[1].hist(norm_img.flatten(), bins=256)
Menampilkan histogram distribusi intensitas piksel dari citra hasil normalisasi pada subplot kedua. Fungsi flatten() mengubah matriks citra menjadi array satu dimensi agar frekuensi tiap intensitas piksel dapat dihitung. Parameter bins=256 menunjukkan jumlah tingkat intensitas grayscale.

• ax[1].set_title("Histogram Setelah Normalisasi")
Memberikan judul pada histogram untuk menjelaskan bahwa grafik menunjukkan distribusi intensitas setelah proses normalisasi.

• ax[1].set_xlabel("Intensitas Pixel")
Memberi label pada sumbu horizontal yang menunjukkan nilai intensitas piksel.

• ax[1].set_ylabel("Jumlah Pixel")
Memberi label pada sumbu vertikal yang menunjukkan jumlah piksel pada setiap tingkat intensitas.

• plt.tight_layout()
Mengatur tata letak subplot secara otomatis agar tidak saling bertumpuk dan terlihat lebih rapi.

• plt.show()
Menampilkan hasil visualisasi citra dan histogram ke layar output.

Setelah citra dinormalisasi, tahap berikutnya adalah melakukan histogram equalization untuk meningkatkan kontras citra secara lebih merata. Proses ini bekerja dengan cara menyebarkan distribusi intensitas piksel berdasarkan fungsi distribusi kumulatif (Cumulative Distribution Function / CDF).

```
hist, bins = np.histogram(norm_img.flatten(), 256, [0,256])

cdf = hist.cumsum()

cdf_normalized = (cdf - cdf.min()) * 255 / (cdf.max() - cdf.min())
cdf_normalized = cdf_normalized.astype(np.uint8)

equalized_img = cdf_normalized[norm_img]
```

• hist, bins = np.histogram(norm_img.flatten(), 256, [0,256])
Baris ini digunakan untuk menghitung histogram citra hasil normalisasi. Fungsi flatten() mengubah matri ks citra menjadi array satu dimensi agar frekuensi kemunculan tiap nilai intensitas dapat dihitung. Parameter 256 menunjukkan jumlah tingkat intensitas grayscale, sedangkan [0,256] menunjukkan rentang nilai piksel.

• cdf = hist.cumsum()
Baris ini menghitung *Cumulative Distribution Function (CDF)*, yaitu jumlah kumulatif dari histogram. Nilai CDF menunjukkan akumulasi frekuensi piksel dari intensitas terendah hingga tertinggi.

• cdf_normalized = (cdf - cdf.min()) * 255 / (cdf.max() - cdf.min())
Baris ini melakukan normalisasi nilai CDF agar berada dalam rentang intensitas grayscale standar (0–255). Proses ini bertujuan untuk membentuk pemetaan baru intensitas piksel agar distribusinya lebih merata.

• cdf_normalized = cdf_normalized.astype(np.uint8)
Baris ini mengubah tipe data hasil normalisasi CDF menjadi unsigned 8-bit integer agar sesuai dengan format citra digital.

• equalized_img = cdf_normalized[norm_img]
Baris ini melakukan proses pemetaan ulang intensitas piksel citra menggunakan nilai CDF yang telah dinormalisasi.

Setelah proses histogram equalization selesai, citra hasil perbaikan kontras kemudian ditampilkan beserta histogram terbarunya untuk melihat perubahan distribusi intensitas piksel secara visual. Pada gambar mulai terlihat adanya perubahan pada citra, terutama pada bagian rambut karakter yang sedikit memutih karena kontras citra mengalami peningkatan.

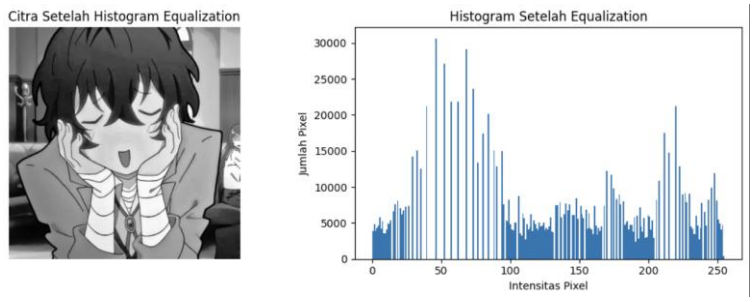
• fig, ax = plt.subplots(1, 2, figsize=(11,4))
Baris ini membuat sebuah figure dengan dua subplot yang disusun dalam satu baris dan dua kolom. Parameter figsize mengatur ukuran tampilan agar citra dan histogram terlihat jelas serta proporsional.

```
fig, ax = plt.subplots(1, 2, figsize=(11,4))

# Subplot 1 — Equalized Image
ax[0].imshow(equalized_img, cmap='gray')
ax[0].set_title("Citra Setelah Histogram Equalization")
ax[0].axis('off')

# Subplot 2 — Histogram Equalized
ax[1].hist(equalized_img.flatten(), bins=256)
ax[1].set_title("Histogram Setelah Equalization")
ax[1].set_xlabel("Intensitas Pixel")
ax[1].set_ylabel("Jumlah Pixel")

plt.tight_layout()
plt.show()
```

• ax[0].imshow(equalized_img, cmap='gray')
Menampilkan citra hasil histogram equalization pada subplot pertama dalam bentuk grayscale sehingga peningkatan kontras dapat terlihat dengan lebih jelas.

• ax[0].set_title("Citra Setelah Histogram Equalization")
Memberikan judul pada subplot pertama untuk menjelaskan bahwa citra yang ditampilkan merupakan citra hasil histogram equalization.

• ax[0].axis('off')
Menyembunyikan sumbu koordinat agar tampilan citra menjadi lebih rapi dan fokus pada visual gambar.



• ax[1].hist(equalized_img.flatten(), bins=256)
Menampilkan histogram distribusi intensitas piksel dari citra hasil equalization pada subplot kedua. Fungsi flatten() mengubah matriks citra menjadi array satu dimensi agar frekuensi tiap nilai piksel dapat dihitung. Parameter bins=256 menunjukkan jumlah tingkat intensitas grayscale.

• ax[1].set_title("Histogram Setelah Equalization")
Memberikan judul pada histogram untuk menjelaskan bahwa grafik menunjukkan distribusi intensitas setelah proses histogram equalization.

• ax[1].set_xlabel("Intensitas Pixel")
Memberi label pada sumbu horizontal yang menunjukkan nilai intensitas piksel.

• ax[1].set_ylabel("Jumlah Pixel")
Memberi label pada sumbu vertikal yang menunjukkan jumlah piksel pada setiap tingkat intensitas.

• plt.tight_layout()
Mengatur tata letak subplot secara otomatis agar tidak saling bertumpuk dan terlihat lebih rapi.

• plt.show()
Menampilkan hasil visualisasi citra dan histogram ke layar output.

Link kode github: https://github.com/sonicmortis/2311532020-Luthfi-Kautsar-Rizata-ImageProcessing/tree/main/Tugas%20Pertemuan%204